Examen Parcial 2 — Trabajo Final de Ingeniería

Universidad Abierta Interamericana · Facultad de Tecnología Informática. Proyecto EventAura — plataforma de venta de entradas con armado personalizado de paquetes. Alumnos: Ambrosino, Máximo · Riggio, Juan Cruz.

Caso de uso seleccionado y alcance de este documento

Se presenta **CU-01 — Comprar paquete,** que realiza el requerimiento RF-05 «Armado de paquetes personalizados». Es el caso de uso core del proyecto por tres motivos: es el requerimiento de mayor esfuerzo estimado (13 puntos sobre 78, el doble que cualquier otro), materializa el diferencial del producto —combinar entrada, hospedaje, traslado, merchandising y experiencia VIP en una sola operación— y es el que más reglas de negocio pone en juego: de las treinta y ocho vigentes, veintidós intervienen en su recorrido.

Sobre la versión de los diagramas. Los diagramas que siguen fueron elaborados contra el código efectivamente implementado y no contra el diseño preliminar de la versión 2.4 del informe. La construcción del software, realizada en diez etapas verificadas, introdujo cambios de modelo que el diseño original no anticipaba. Las diferencias no se ocultan: se documentan una por una en el anexo final, con su justificación.

a. Diagrama de casos de uso

El actor principal es el Cliente autenticado. La pasarela de pago participa como actor secundario: es un componente simulado que devuelve aprobado o rechazado según una regla fija, sin acceso a la red, conforme al alcance académico del proyecto.

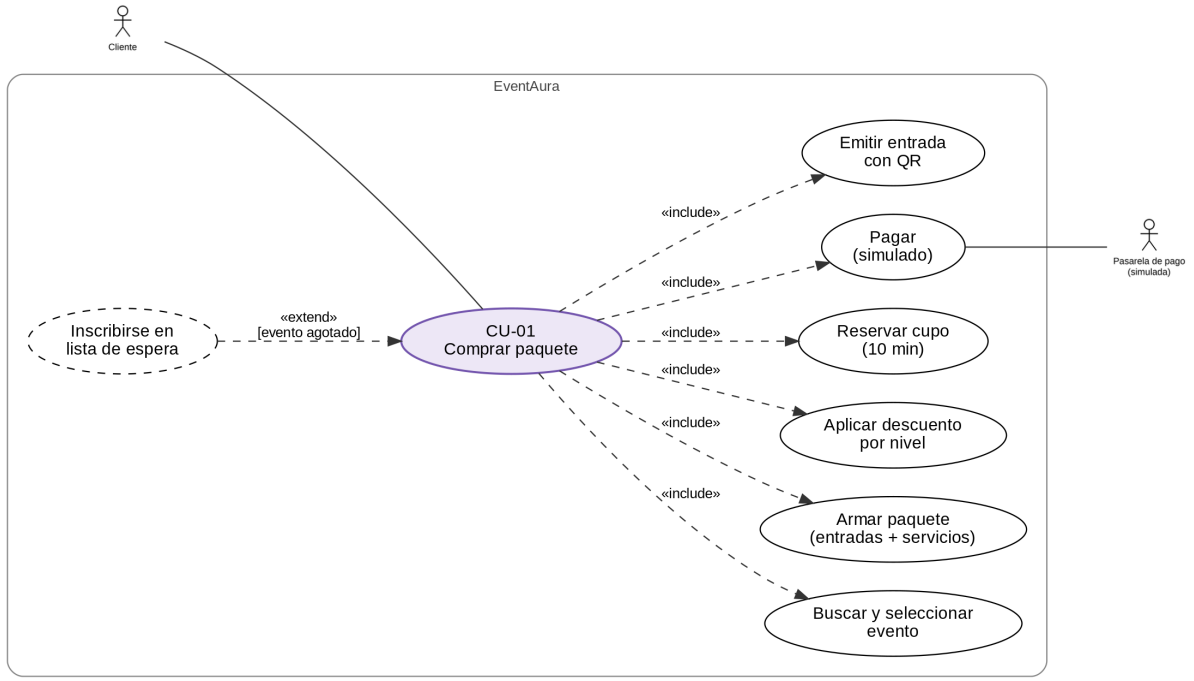


Figura 1 — Casos de uso: CU-01 y sus relaciones

Las seis relaciones «include» corresponden a pasos que siempre se ejecutan dentro del caso de uso. La relación «extend» hacia la lista de espera se activa únicamente cuando el evento está agotado según RN-10, es decir, cuando ningún sector conserva capacidad sin vender.

Especificación de CU-01

Campo	Contenido
Identificador	CU-01 — Comprar paquete
Actor principal	Cliente (autenticado)
Actor secundario	Pasarela de pago simulada
Requerimiento	RF-05 (core) · RF-04 · RF-06 · RF-07 · RF-08
Precondiciones	El cliente tiene sesión iniciada. Existe al menos un evento publicado con fecha futura y cupo disponible.
Postcondiciones	Existe una fila en Compras en estado Confirmada, una fila en Entradas por cada entrada adquirida con su código QR único, y un registro del intento de pago en Pagos. El nivel de fidelidad del cliente queda recalculado y rige desde la operación siguiente.

Flujo principal

1. El cliente selecciona un evento del listado y abre la pantalla de compra.
2. El sistema libera las reservas caducadas y muestra el cupo disponible por sector, junto con el nivel de fidelidad vigente del cliente y el porcentaje que le corresponde.
3. El cliente indica la cantidad de entradas por sector y marca los servicios complementarios que

desea incorporar.

4. El sistema recalcula el resumen en cada cambio: base de entradas, descuento por nivel, subtotal, servicios, cargo por servicio, seguro y total.

5. El cliente confirma. El sistema valida el paquete, registra la compra en estado Pendiente y reserva el cupo por diez minutos.

6. El cliente ingresa los datos de la tarjeta en la pantalla de pago.

7. El sistema valida el formato, solicita el cobro a la pasarela simulada y registra el intento con su resultado.

8. Si el pago se aprueba, el sistema confirma la compra, emite una entrada con código QR único por cada unidad adquirida, graba las pólizas de seguro si corresponde y reevalúa el nivel del cliente.

9. El sistema genera un PDF por entrada y muestra la pantalla de confirmación con el número de pedido y el desglose de lo pagado.

Flujos alternativos

FA1 — Paquete sin entradas. Si el cliente intenta confirmar habiendo elegido solo servicios, el sistema rechaza la operación citando RN-13: los servicios no se venden sueltos.

FA2 — Cupo insuficiente. Si entre el armado y la confirmación otro cliente agotó el sector, la reserva falla y el sistema informa cuántas entradas quedan (RN-08).

FA3 — Pago rechazado. El sistema registra el intento, devuelve el cupo en el acto (RN-32) y deja la compra pendiente para que el cliente reintente con otra tarjeta dentro del plazo original.

FA4 — Reserva vencida. Si transcurren diez minutos sin confirmar, la compra pasa a Vencida y el cupo vuelve a estar disponible (RN-15).

FA5 — Evento agotado. Si el evento no conserva cupo en ningún sector, la pantalla ofrece la inscripción en la lista de espera en lugar de la grilla de sectores (RN-10).

b. Diagrama de dominio

El modelo distingue tres grupos: las entidades persistentes, que tienen tabla propia; el paquete, que vive únicamente en memoria durante el armado; y los parámetros de configuración, que externalizan los valores numéricos de las reglas.

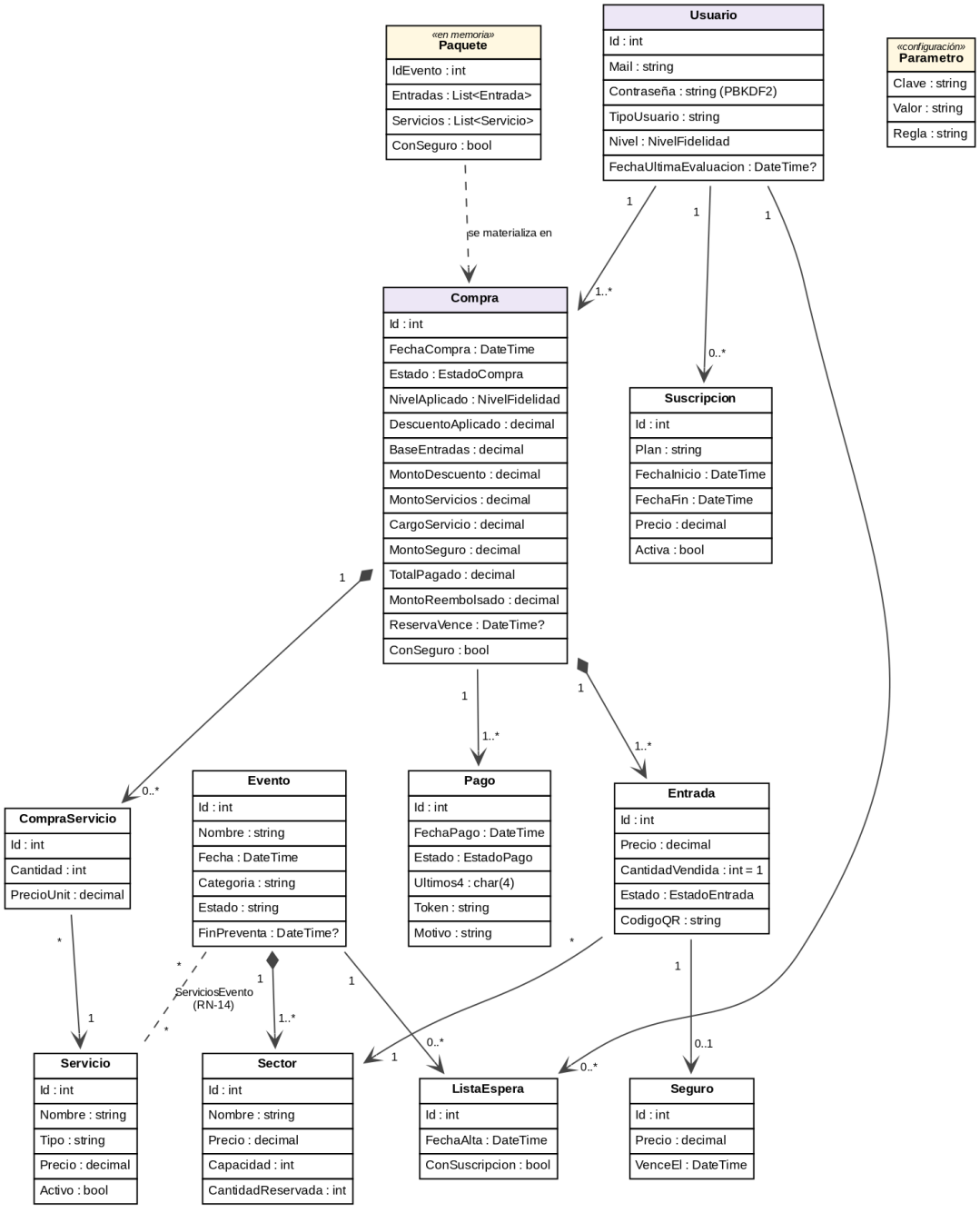


Figura 2 — Modelo de dominio según la implementación

Decisiones de modelado que conviene justificar

Paquete no tiene tabla. Es una estructura en memoria que agrupa lo que el cliente está armando. Solo se materializa al iniciar la compra, momento en que se convierte en una fila de Compras con sus filas de Entradas y CompraServicios. Mientras el cliente no confirma, no hay nada escrito en la base.

Compra es una entidad propia y no un atributo de Entrada. RN-19 cuenta operaciones, no entradas: sin agrupar, un cliente que compra tres entradas en una transacción ascendería de nivel al triple de velocidad. Además RN-16 y RN-22 exigen congelar el nivel y el porcentaje aplicados, y esos son atributos de la operación.

Una fila de Entradas por entrada. RN-18 exige un código QR único por entrada, válido para un único ingreso, y la columna CodigoQR es una por fila. Lo mismo vale para Seguros, que lleva una póliza por entrada. Por eso CantidadVendida vale siempre uno.

Un solo catálogo de servicios con atributo Tipo. El diseño preliminar proponía una jerarquía con ProductoServicio abstracto y una subclase por tipo. Se descartó: las cuatro variantes no agregan comportamiento propio, solo difieren en un valor, y la jerarquía obligaría a una tabla por subclase sin beneficio alguno.

c. Prototipo del formulario de carga

El formulario de carga del caso de uso es la pantalla de armado del paquete. El prototipo que sigue está dibujado a escala a partir de las coordenadas declaradas en Visual/Cliente/FrmCompraCliente.Designer.cs, de modo que refleja la disposición real de los controles y no una maqueta idealizada.

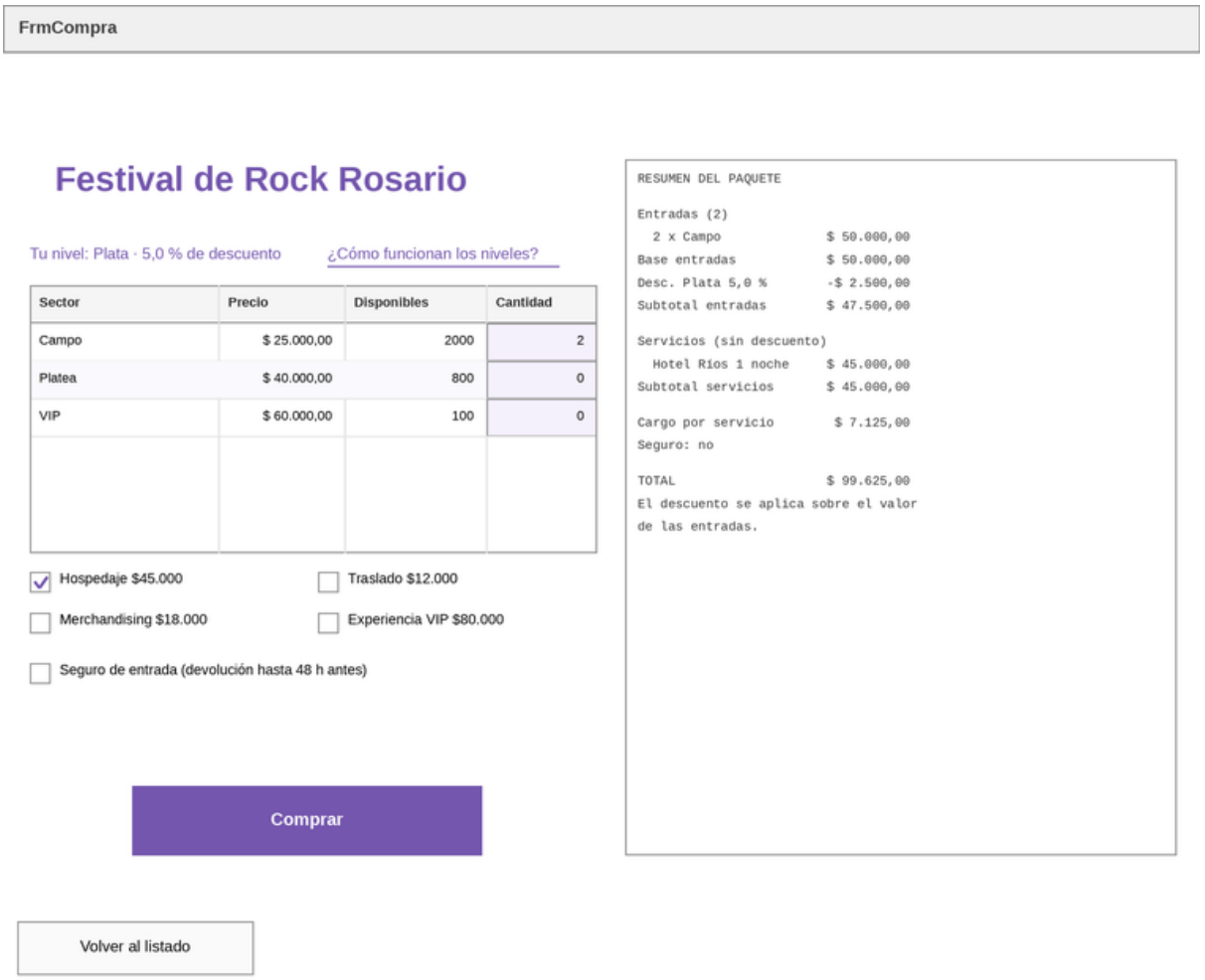


Figura 3 — FrmCompraCliente: armado del paquete

La pantalla resuelve tres hallazgos del informe de usabilidad. La columna «Disponibles» responde a H-03, que señalaba que el selector no informaba el cupo restante. El enlace «¿Cómo funcionan los niveles?» responde a H-05. Y la leyenda bajo el total, «El descuento se aplica sobre el valor de las entradas», responde a H-01, el hallazgo más repetido entre los participantes.

Los importes del resumen corresponden al caso de verificación numérica del proyecto: cliente de nivel Plata, dos entradas de Campo a $25.000 y un hospedaje de $45.000, con un total de $99.625. Ese número atraviesa el instructivo, la especificación del descuento y los casos de prueba, y funciona como control cruzado de la implementación.

d. Arquitectura de la solución

La solución se organiza en cuatro capas con dependencias unidireccionales. Cada capa es un proyecto independiente de la solución, de modo que la separación no es una convención de carpetas sino una restricción que el compilador hace cumplir: si Visual intentara abrir una conexión, no compilaría, porque no referencia el proveedor de datos.

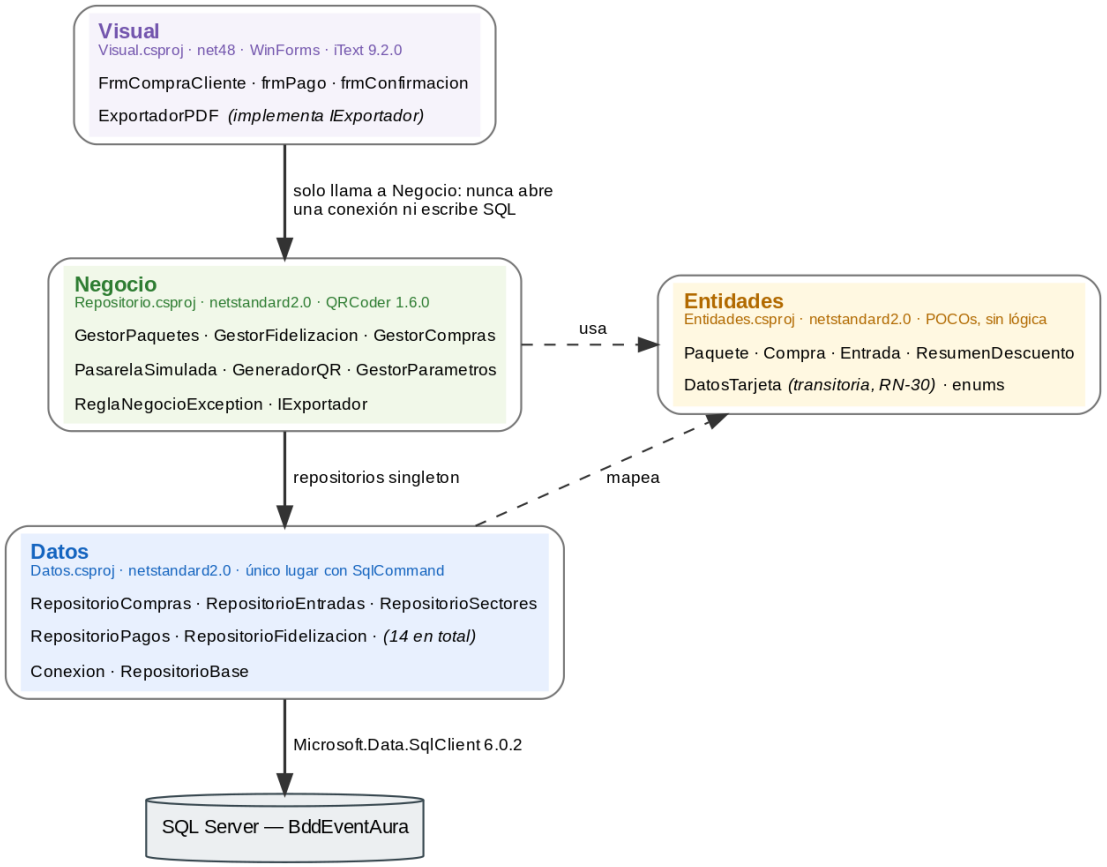


Figura 4 — Arquitectura en cuatro capas

Reglas de dependencia

- Visual nunca abre una conexión ni escribe SQL. Solo invoca a Negocio.
- Negocio no referencia System.Windows.Forms. No muestra mensajes: devuelve resultados o lanza excepciones de dominio que la capa de presentación traduce.
- Datos es el único lugar donde existen SqlCommand, SqlConnection y texto SQL.
- Entidades no referencia a nadie: son objetos con propiedades automáticas y sin lógica.
- Todo acceso a la base usa consultas parametrizadas; nunca concatenación de cadenas.

Consecuencia práctica de la elección de framework. Entidades, Negocio y Datos compilan contra netstandard2.0 y suman 5.374 líneas. Solo Visual, con 6.933 líneas, está atado a WinForms. Si el proyecto migrara a una interfaz web, las tres primeras capas se reutilizarían sin modificación desde una API, porque ningún gestor conoce la presentación. La excepción ReglaNegocioException expone el identificador de la regla incumplida, que en un servicio HTTP se traduce directamente al cuerpo de

una respuesta 400.

e. Especificación de los modelos de datos

Se detallan las tablas que intervienen en CU-01. Los tipos son los efectivamente declarados en BaseDatos/01-esquema.sql. Todos los importes son DECIMAL(12,2): el proyecto prohíbe float y double para valores monetarios.

Compras — la operación

Columna	Tipo	Descripción
ID	INT IDENTITY	Clave primaria.
ID_Usuario	INT NOT NULL	FK a Usuario. Quién compró.
FechaCompra	DATETIME NOT NULL	Momento de inicio. Fija el plazo de la reserva.
Estado	VARCHAR(12)	Pendiente | Confirmada | Reembolsada | Vencida.
NivelAplicado	VARCHAR(8)	Nivel congelado al iniciar (RN-22).
DescuentoAplicado	DECIMAL(4,3)	Porcentaje congelado (RN-16).
BaseEntradas	DECIMAL(12,2)	Suma de las entradas, antes del descuento.
MontoDescuento	DECIMAL(12,2)	Descuento por nivel, solo sobre entradas (RN-21).
MontoServicios	DECIMAL(12,2)	Servicios complementarios, sin descuento.
CargoServicio	DECIMAL(12,2)	15 % sobre el subtotal de entradas ya descontado.
MontoSeguro	DECIMAL(12,2)	Prima total. Cero si el cliente tiene suscripción.
ConSeguro	BIT NOT NULL	Si contrató el seguro. Necesaria porque con suscripción la prima vale cero y el monto no alcanza para saberlo.
TotalPagado	DECIMAL(12,2)	Lo que se cobró. No cambia ante una devolución.
MontoReembolsado	DECIMAL(12,2)	Acumulado devuelto (RN-29).
ReservaVence	DATETIME NULL	Con valor, la compra retiene cupo; en NULL ya lo devolvió.

Invariante de la fila. Para toda compra se cumple que BaseEntradas − MontoDescuento + MontoServicios + CargoServicio + MontoSeguro = TotalPagado. Es la condición que se verifica en las pruebas automatizadas sobre el total de las filas, y la razón por la que MontoReembolsado es una columna aparte en lugar de restarse del total pagado.

Entradas — una fila por entrada vendida

Columna	Tipo	Descripción
ID	INT IDENTITY	Clave primaria.
ID_Compra	INT NULL	FK a Compras. Agrupa las entradas de una operación.
ID_Sector	INT NULL	FK a Sectores. Sin esta columna RN-08 no se puede evaluar.
Precio	DECIMAL(12,2)	Precio unitario congelado al armar el paquete (RN-16).
CantidadVendida	INT	Vale siempre 1: una fila por entrada.
Estado	VARCHAR(12)	Pendiente | Confirmada | Reembolsada | Vencida.
CodigoQR	VARCHAR(64)	Único e irrepetible (RN-18). Índice único filtrado.

Restricciones que la base garantiza por estructura

Cinco reglas no se delegan al código: se imponen en el esquema. El motivo es que una validación en

memoria no resiste dos operaciones simultáneas, y porque una regla escrita solo en C# deja de cumplirse ante cualquier escritura que no pase por la aplicación.

Regla	Restricción	Qué impide
RN-01	UQ_Usuario_Mail	Dos cuentas con el mismo correo.
RN-08	FK_Entradas_Sectores	Una entrada sin sector: la regla quedaría inevaluable.
RN-18	UX_Entradas_CodigoQR	Un QR repetido habilitaría dos ingresos.
RN-25	UX_Suscripcion_Activa	Dos suscripciones activas por usuario (índice filtrado sobre Activa = 1).
RN-36	UX_ListaEspera	Inscribirse dos veces en el mismo evento.

Consulta de cupo disponible

El control de disponibilidad resuelve el conflicto entre RN-08 y RN-15: lo disponible descuenta tanto lo vendido como lo reservado y aún vigente.

```
SELECT  s.Capacidad
      - ISNULL((SELECT SUM(e.CantidadVendida)
                FROM   Entradas e
                WHERE  e.ID_Sector = s.ID
                  AND  e.Estado    = 'Confirmada'), 0)
      - s.CantidadReservada                        AS Disponible
FROM    Sectores s
WHERE   s.ID = @IdSector;
```

La reserva se toma con un UPDATE condicional en una sola sentencia, cuya cláusula WHERE repite ese cálculo. Así la verificación y la escritura son atómicas: dos compras simultáneas sobre el último lugar no pueden sobrevender, porque la segunda no afecta ninguna fila y la transacción se deshace.

f. Diagrama de robustez

El diagrama aplica la notación ICONIX y respeta sus reglas de conexión: el actor solo toca objetos de interfaz; los de interfaz solo hablan con controladores; y únicamente los controladores acceden a entidades. Esa disciplina es la que se traduce luego en las cuatro capas de la arquitectura.

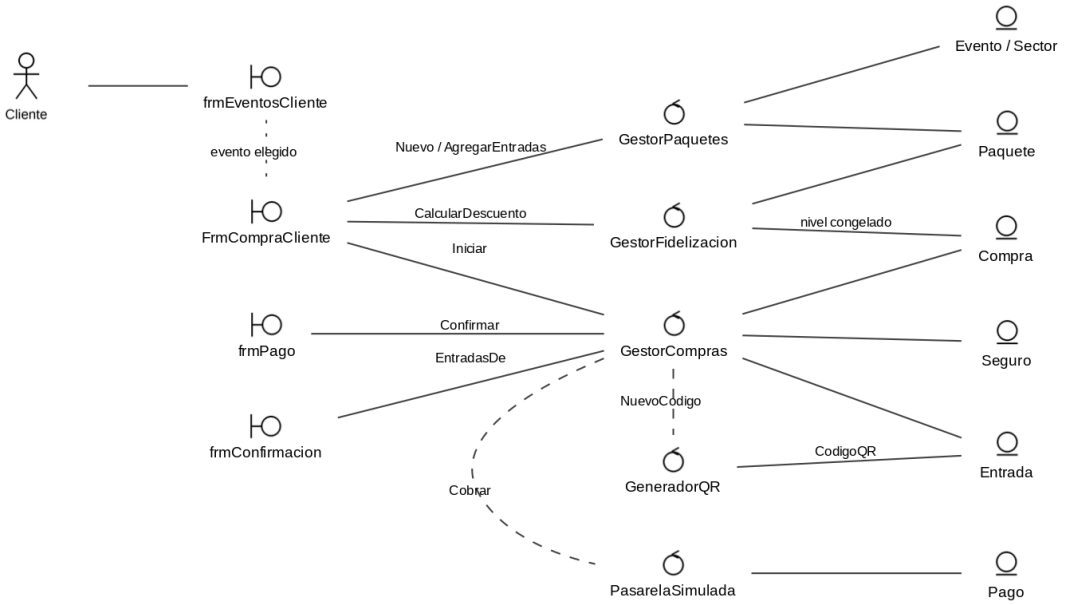


Figura 5 — Robustez de CU-01 (ICONIX)

La correspondencia con el código es directa: los objetos de interfaz son los formularios del proyecto Visual, los controladores son los gestores del proyecto Negocio, y las entidades son las clases del proyecto Entidades que los repositorios materializan. No hay ninguna flecha entre un objeto de interfaz y una entidad, que es exactamente la regla que impide que un formulario escriba SQL.

g. Especificación de los formularios de carga

FrmCompraCliente — armado del paquete

Control	Tipo	Entrada y validación
dgvSectores	DataGridView	Una fila por sector. Solo la columna Cantidad es editable; acepta enteros no negativos. Si el valor supera el cupo se rechaza con RN-08 y la celda vuelve al valor anterior.
chkHospedaje, chkTraslado, chkMerch, chkVIP	CheckBox	Un servicio por tipo. Se habilitan solo si el evento los tiene asociados (RN-14); si no, el texto indica «no disponible» y el control queda deshabilitado.
chkSeguro	CheckBox	Opcional. Solo puede contratarse durante la compra (RN-28).
lblResumen	Label	Solo lectura. Recalculado ante cualquier cambio.
btnComprar	Button	Valida el paquete completo y reserva el cupo. Si no hay seguro, pide confirmación explícita (H-04).

Validaciones de negocio que se disparan desde este formulario

RN-12. Máximo seis entradas por operación, sumando todos los sectores.

RN-13. El paquete debe contener al menos una entrada.

RN-08. La cantidad pedida no supera el cupo disponible del sector.

RN-09. El evento debe estar publicado y con fecha futura.

RN-39. Durante la preventa, solo compra un cliente con suscripción activa.

frmPago — datos de la tarjeta

Control	Tipo	Entrada y validación
txtTitular	TextBox	Obligatorio, hasta 60 caracteres.
txtNumero	TextBox	Entre 13 y 19 dígitos (ISO/IEC 7812). Se admiten espacios y guiones, que se descartan.
txtVencimiento	TextBox	Formato MM/AA. Se rechaza una tarjeta vencida; vale hasta el último día del mes impreso.
txtCvv	TextBox	Tres o cuatro dígitos. Se muestra enmascarado y se limpia tras cada intento.
lblTemporizador	Label	Cuenta regresiva de la reserva, siempre visible (H-02). Al llegar a cero libera el cupo y cierra la pantalla.

Tratamiento de los datos sensibles. La clase DatosTarjeta es transitoria: no tiene tabla y su instancia vive solo mientras dura el cobro. De la tarjeta se persisten únicamente los últimos cuatro dígitos y un token ficticio, y una restricción CHECK en la columna Ultimos4 impide por estructura que se cuele otra cosa. El número completo nunca llega a la capa de datos (RN-30).

h. Diagramas de secuencia

El caso de uso se presenta en dos diagramas porque tiene dos transacciones distintas, separadas por la interacción del cliente con la pantalla de pago. Entre una y otra la compra queda persistida en estado Pendiente, de modo que la reserva sobrevive incluso si la aplicación se cierra.

h.1 Armado del paquete y reserva del cupo

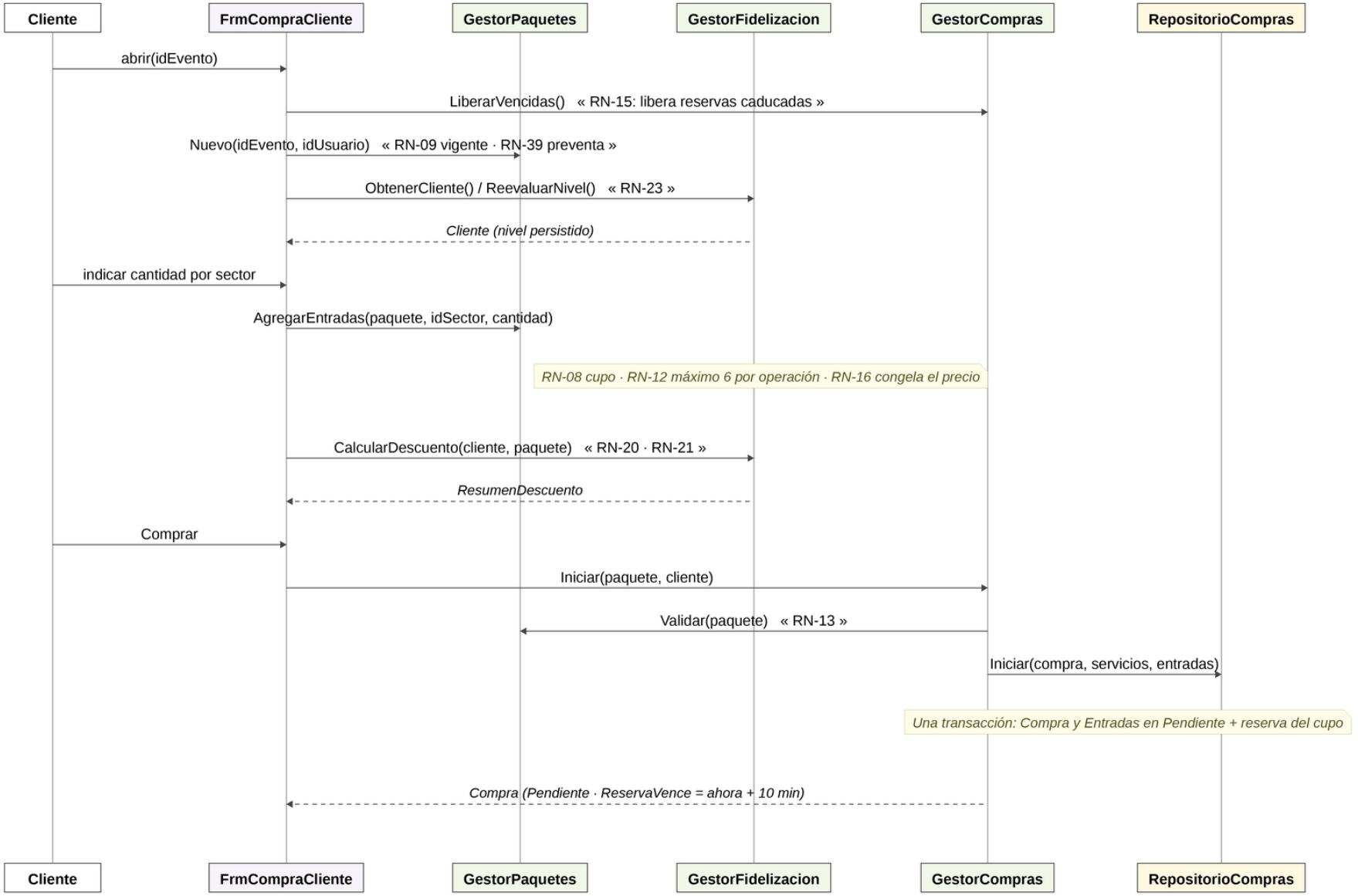


Figura 6 — Secuencia: del armado a la reserva

Dos detalles que el diagrama hace explícitos. El primero es que el descuento se calcula dos veces: una al mostrar el resumen en pantalla y otra al iniciar la compra. No es redundancia: el nivel del cliente se lee de la entidad y los precios ya están congelados en el paquete, de modo que ambos cálculos coinciden y el segundo es el que queda grabado. El segundo es que la liberación de reservas vencidas ocurre al abrir la pantalla: no hay proceso en segundo plano, así que la limpieza es perezosa.

h.2 Pago simulado, confirmación y emisión

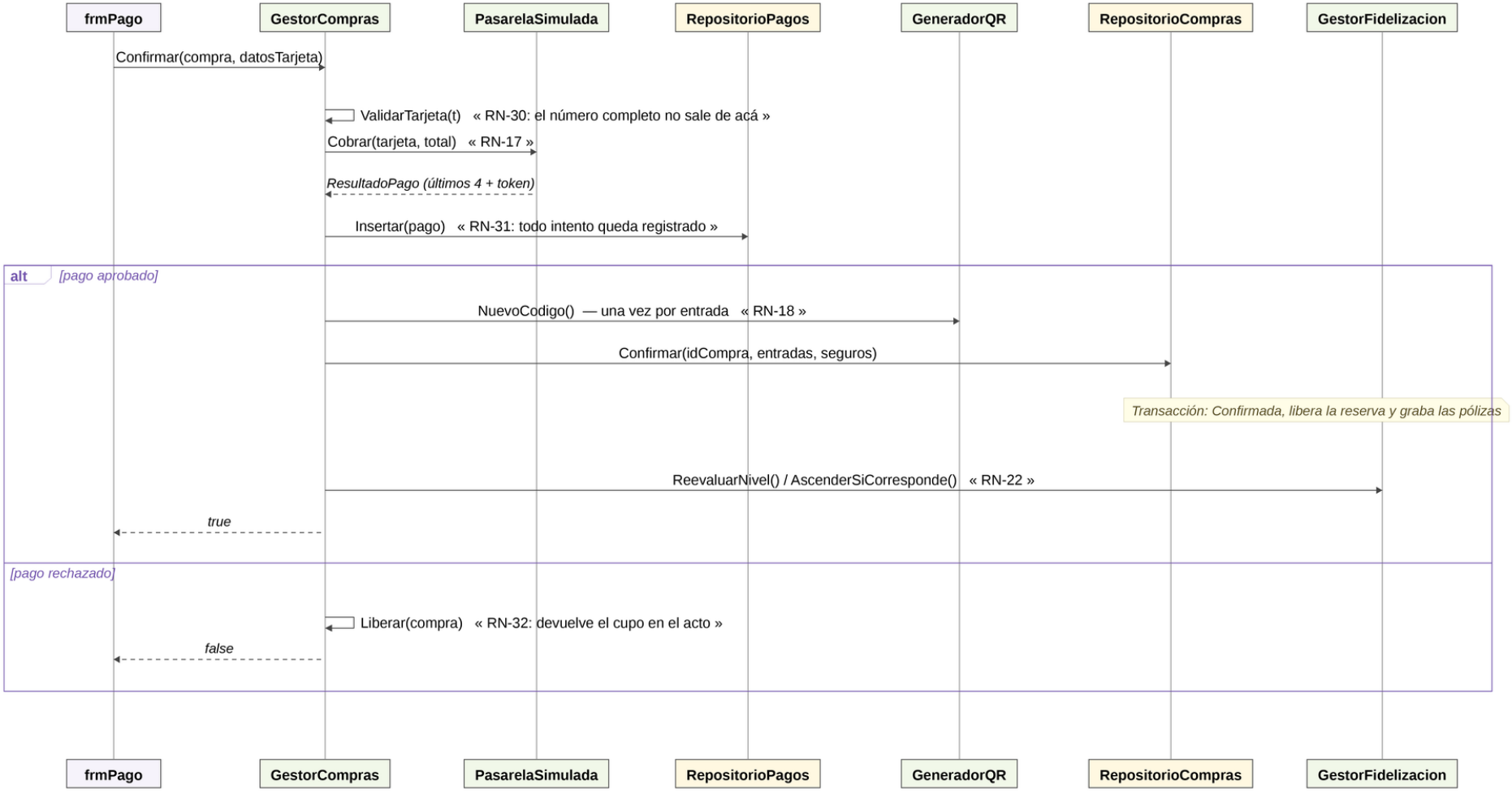


Figura 7 — Secuencia: del cobro a la entrada emitida

El fragmento alternativo distingue las dos ramas. En ambas el intento queda registrado en Pagos antes de decidir, que es lo que exige RN-31: se registra la operación de pago con su estado resultante, haya sido aprobada o no. En la rama de rechazo el cupo se devuelve de inmediato y la compra permanece en Pendiente, de modo que el cliente puede reintentar con otra tarjeta sin rearmar el paquete; el plazo, eso sí, sigue corriendo desde que la compra se inició.

i. Implementación de patrones de diseño

Se documentan tres patrones efectivamente implementados, con su ubicación en el código y el problema concreto que resuelven. Se omiten deliberadamente los patrones que el proyecto menciona como previstos pero no construidos, para no atribuir al sistema algo que no tiene.

Singleton — instancia única por gestor y repositorio

Dónde. Veintinueve clases: los catorce repositorios de Datos y los quince gestores de Negocio. La implementación sigue el mismo molde en todas, con constructor privado y propiedad estática Instance de inicialización perezosa.

Qué resuelve. Dos cosas distintas. En los repositorios, concentra la construcción de conexiones en un punto único. En GestorParametros es el patrón el que hace viable la regla de que ningún valor numérico esté escrito en el código: la tabla Parametros se lee una sola vez por ejecución y queda en memoria, de modo que consultar el porcentaje de descuento no cuesta una consulta por invocación.

```
public sealed class GestorFidelizacion
{
    private static GestorFidelizacion _instance;

    public static GestorFidelizacion Instance
    {
        get
        {
            if (_instance == null) { _instance = new GestorFidelizacion(); }
            return _instance;
        }
    }

    private GestorFidelizacion() { }
}
```

Limitación asumida. La inicialización no es segura ante hilos. La aplicación es de escritorio y de un solo hilo de interfaz, de modo que la condición de carrera no puede darse; en un servicio web habría que sincronizar el acceso o delegar la vida del objeto a un contenedor de inyección.

Strategy — formatos de exportación intercambiables

Dónde. La interfaz IExportador se declara en Negocio y ExportadorPDF la implementa en Visual. GestorReportes recibe la estrategia como parámetro y no conoce ninguna implementación concreta.

```
public interface IExportador
{
    string Exportar(Reporte r, DataTable datos, string ruta);
}

// En GestorReportes: la estrategia entra por parametro.
public string Exportar(Reporte r, IExportador exp, string ruta)
{
    return exp.Exportar(r, Ejecutar(r), ruta);
}
```

Qué resuelve. Separa qué se informa de cómo se entrega. Agregar una exportación a Excel es una clase nueva que implemente la interfaz, sin tocar ninguno de los cinco reportes. Pero el beneficio decisivo es de arquitectura: iText es una librería de presentación y Negocio no la referencia. Sin esta interfaz, generar el PDF desde la capa de negocio habría obligado a que Negocio dependiera de iText, rompiendo la regla de capas.

Template Method — esqueleto común de los reportes

Dónde. La clase abstracta Reporte define el algoritmo y delega tres pasos en sus cinco subclases concretas.

```
public abstract class Reporte
{
    public abstract string[]  Encabezados();
    public abstract DataTable Obtener(RepositorioReportes repo);
    public abstract string    Lectura(DataTable datos);
}
```

Qué resuelve. Los cinco reportes comparten la estructura —encabezado, filtros aplicados, tabla con totales, párrafo interpretativo y pie— y difieren solo en la consulta, en los nombres de las columnas y en cómo se redacta la lectura. El exportador recorre esa estructura sin saber qué reporte está componiendo. Un sexto reporte es una subclase nueva.

Patrones de arquitectura

Repository. Cada repositorio encapsula el acceso a un agregado y expone métodos del dominio —ObtenerPorId, ContarComprasConfirmadas— en lugar de consultas. La capa de negocio nunca ve una sentencia SQL.

Capas. La separación en cuatro proyectos con dependencias unidireccionales, descrita en el punto d.

j. Gestión de configuración

Datos demo

El juego de datos está versionado como código, en tres guiones ejecutables en orden. Ninguno depende del estado previo de la base, de modo que cualquier integrante del equipo reconstruye un entorno idéntico en menos de un minuto.

Guion	Qué carga	Para qué
01-esquema.sql	Las tablas, claves, índices y restricciones.	Crea el esquema completo. Incluye las tablas preexistentes con guardas, de modo que sirve tanto para una base vacía como para una ya poblada.
02-semilla.sql	Diecisiete parámetros, cuatro usuarios —uno por nivel más el administrador—, un local, tres eventos, sus sectores, cuatro servicios y un historial de compras coherente con el nivel de cada cliente.	Es el mínimo para recorrer los casos de uso. El historial está construido para que cada usuario quede efectivamente en su nivel según RN-19.
03-datos-reporte.sql	1.178 usuarios, 2.800 compras y 4.480 entradas distribuidas en cinco eventos.	Da volumen a los reportes. Se ejecuta aparte y puede correrse dos veces seguidas porque empieza borrando lo que él mismo cargó.

Por qué los datos demo son código y no un respaldo. Un archivo de respaldo binario no se puede revisar en un cambio, no se puede comparar entre versiones y queda desactualizado en cuanto el esquema cambia. Un guion versionado se lee, se discute y evoluciona junto al esquema que lo acompaña.

Control de versiones

El repositorio está en Git con remoto en GitHub. La estrategia de commits es deliberada: una unidad de trabajo verificable por commit, con el mensaje identificando la etapa.

Prefijo del mensaje	Qué agrupa	Commits
Seccion N: …	Una etapa de construcción con su verificación. Las etapas 4 y 5 ocupan dos commits cada una porque se cerraron en dos entregas.	12
Correccion …	Defectos detectados al verificar una etapa ya cerrada. Agrupan nueve correcciones identificadas como C-1 a C-9.	3
Docs: …	Cambios de documentación sin efecto sobre el código.	5
Esquema: …	Correcciones de integridad sobre el modelo de datos.	2
Otros	Preparación del repositorio, README e instructivos.	6

Veintiocho commits en total. La relación entre correcciones y commits no es uno a uno: las siete primeras correcciones se agruparon en un solo commit por tratarse de la misma revisión, mientras que C-8 y C-9 se aislaron porque cada una resolvía un defecto independiente detectado después.

El valor de esta disciplina se verificó en la práctica. Al cerrar la etapa nueve se detectó que un cliente con suscripción quedaba sin póliza de seguro, en silencio. Para demostrar que el defecto era real y no una interpretación, se reconstruyó el binario de la etapa anterior en un árbol de trabajo separado y se ejecutó contra él la misma prueba: cero pólizas antes del cambio, tres después. Eso solo es posible si cada etapa quedó en un commit propio y compilable.

Trazabilidad entre el código y los requerimientos

Cada regla de negocio aparece citada por su identificador en el comentario del método que la implementa. La consecuencia es que el recorrido entre el requerimiento y la línea de código que lo cumple es una búsqueda de texto, en cualquiera de los dos sentidos.

Anexo — Deriva entre el diseño preliminar y la implementación

Los diagramas de la versión 2.4 del informe se elaboraron antes de construir el software. La implementación obligó a revisarlos. Se consignan las diferencias con su motivo, porque un diagrama que contradice al código es peor que no tenerlo.

Diseño preliminar	Implementación	Motivo del cambio
Cliente acumula puntos; NivelUsuario define umbrales de puntos.	No hay puntos. El nivel se deriva de contar compras confirmadas en una ventana móvil de doce meses.	Un saldo de puntos solo crece y vuelve vitalicio el beneficio. La ventana móvil hace que el nivel pueda bajar solo, lo que acota el costo del programa. Se analizó en la sección de factibilidad económica.
Carrito como entidad persistente.	Paquete en memoria, sin tabla.	Lo que el cliente arma antes de confirmar no es información que el negocio necesite conservar. Persistirlo habría obligado a limpiar carritos abandonados.
ProductoServicio abstracto con subclases Hospedaje, Traslado y Merchandising.	Una tabla Servicios con atributo Tipo.	Las variantes no agregan comportamiento, solo un valor distinto.
Entrada con fila y asiento.	Entrada con sector, sin numeración.	Ningún requerimiento pide asiento numerado; el cupo se controla por sector (RN-08).
Estados de la entrada: Disponible, Reservada, Vendida, Cancelada.	Pendiente, Confirmada, Reembolsada, Vencida.	Una reserva caducada no es una cancelación: se distingue para conservar el rastro del intento que exige RN-31.
Importes declarados como float.	decimal en todo el sistema.	float introduce error de representación en operaciones monetarias.
Sin tablas de pagos, seguros, suscripciones ni parámetros.	Pagos, Seguros, Suscripciones, CompraServicios y Parametros.	RN-30 y RN-31 no tenían dónde registrarse, y los valores de las reglas estaban escritos en el código.
Controladores denominados ControladorCompra y ServicioCompra.	GestorPaquetes, GestorFidelizacion y GestorCompras.	La responsabilidad quedó repartida según el momento del ciclo de vida: armar, calcular y cerrar la operación.

La misma revisión alcanzó al anexo A02 del informe, cuya consulta de ejemplo para el primer reporte no era ejecutable contra el esquema real: nombraba una columna inexistente, multiplicaba las filas por un producto cartesiano y no filtraba por estado. La versión corregida y el detalle de los tres errores están documentados en el repositorio.